Hipstraw Method Compiler — Architecture Specification

Oct 28, 2026 · @Ashish

The Hipstraw Method Compiler is the G3 (Adaptive Launch Methods) addition to the Hipstraw Faculty Harness. It does not introduce launch tactics: those exist from G0 as versioned Moves and staged Lines that the Market or Campaign Manager chooses, with every use recorded in the tactic record (C10). The compiler turns those Lines into validated graphs with branches, so they can be compared under controlled conditions and selected on evidence. It compiles launch tactics only, and every compiled Line ends in a Review whose proposed Position a human accepts [P].

Status, scope and how it relates to the Zimplii compiler

This is an architecture-level specification for review. It is written against Hipstraw Faculty Harness — Evolutionary Architecture ("HA") and follows the structure of the Zimplii Method Compiler — Architecture Specification v2 ("ZMC"). There is no earlier Hipstraw compiler spec. Where this document is silent, HA applies.

What differs from the Zimplii pattern:

Area	Zimplii compiler (ZMC)	Hipstraw compiler
Product	Third-Party Credential Review	Continuous Campaign Strategy & Readiness Testing; compiles launch tactics only [S: HA §1]
Place in the path	G3, after an optional monitoring generation	G3, after G2 Persistent Launch Orchestration, which Hipstraw requires [S: HA §10]
A Move's expected effect	Evidence-class transition on named claims	Change in a Position: claim verification, blocker and hypothesis states (section 3)
Effect classes	E0–E4, E4 payment-related and conditional	E0–E3; E3 publishes, sends, spends, contacts or changes budgets [S: HA §5]
Exit of every Line	Hand to review; output a credential position	Hand to Review; output a proposed successor Position for human acceptance [S: HA §5]

Area	Zimplii compiler (ZMC)	Hipstraw compiler
Outcome evidence	Verified-claim precision	Judgment quality first; campaign outcomes only with stated confounders [S: HS §12]
Runtime	Plan interpreter replaces the stage runner	Same, carried on G2's durable jobs and triggers (section 6)
Trials	Replay, shadow, consented assignment	Adds simulated trials, which never count as field evidence [S: HS §9]

Sources: HA = Hipstraw Faculty Harness — Evolutionary Architecture (its sections, C1–C10, HR and HQ IDs); HS = Hipstraw Faculty Harness Architecture & Evolutionary Implementation Strategy v0.1, 28 October 2026; ZMC = the Zimplii compiler spec v2, used for structure and transferable mechanisms.

Labels: [U] product owner decision · **[S]** source document · **[P]** proposed here · **[A]** assumption · **[O]** open (section 10).

1. Purpose and entry conditions

In HA, Managers already choose Lines and their deviations are recorded, but Lines cannot branch and cannot be compared fairly, because Managers choose them for different kinds of launch [S: HA §11]. The compiler closes exactly those two gaps: a Line can encode the judgments Managers keep making by hand, and Lines can be compared under the controlled conditions observation cannot provide [P].

What the compiler adds to launch testing

- **Branches.** A deviation Managers make again and again, such as "if campaign metrics are stale, request evidence and re-verify", becomes a typed branch in the Line.
- **Safety by construction.** Every Line is checked against scope, authority and honesty rules before it can run.
- **Controlled comparison.** Compiled Lines can be replayed, simulated, shadowed and assigned in consented trials, with their qualification recorded [S: HS §9 G3].
- **Selection on evidence.** The Line Selector offers only Lines qualified for the launch type, within explicit applicability bounds [S: HS §9.1].

What it never does

It never plans steps with a model, never executes, grants authority or qualifies anything by itself [S: HS §8.1; P]. It never writes a success forecast, never treats a Delta as causal proof, and never compiles campaign execution, channel or budget management workflows [S: HA §1, HR02, HR03].

Entry conditions, with the evidence that shows each is met

The six conditions follow the Zimplii pattern [S: ZMC §1], restated for launch testing; HA's tactic record turns most of them into measurable checks [P].

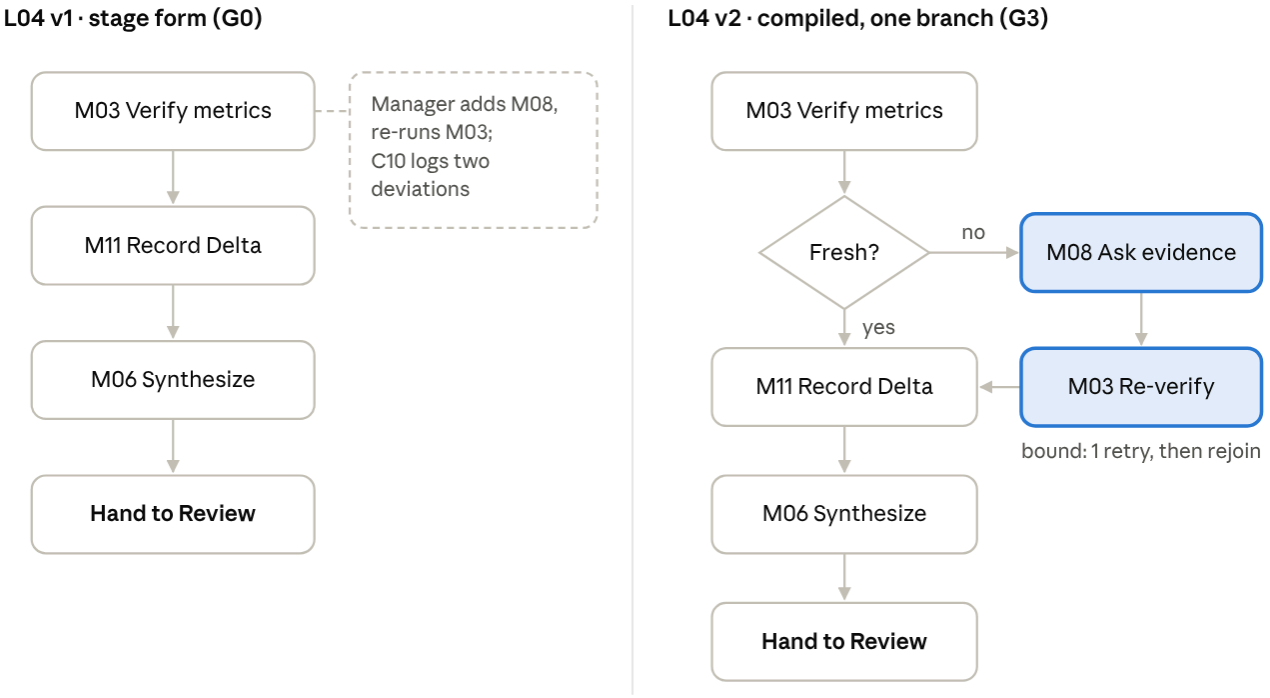
#	Condition	Evidence that it holds
1	G1 and G2 are stable	Correction rate per accepted Review falling across releases; G2 triggering, idempotency and recovery validated with live integrations [S: HA §10, §12]
2	A real choice between Lines exists	Tactic records show, for one launch type, either two Lines in regular use or one stage with a persistently high deviation rate
3	An outcome measure exists	Judgment quality at minimum: missed blockers and evidence-supported findings [S: HS §12]; campaign-outcome measures open [O]
4	Enough eligible launches	Count of launches with scope-3 data rights and complete tactic records for that launch type, above a threshold [O]
5	Authority for effectful steps	E3 Moves only with a mandate channel scope and scoped Sponsor approval, re-checked at dispatch [S: HA §5, §9]
6	Applicability facts recorded	Launch type, market, audience, authorized channels and distance to Day Zero on every Position snapshot [P]

If condition 2 never holds, because one Line per launch type works and Managers rarely deviate, the compiler is not built. That is a valid result: a simpler mechanism wins unless the sophisticated one adds material value [S: HS §10].

2. From stage Lines to compiled Lines

The compiler's first job is to take the Lines Managers already use and represent them exactly, with no change in behaviour. Only then does it add branches, and only where tactic records show Managers repeatedly making the same judgment by hand [P].

A recurring Manager judgment becomes one bounded branch



The branch predicate reads an observation state; illustrative until real deviation patterns exist

Line L04 · stage form vs compiled form with one branch

On the left, L04 Live checkpoint runs three stages; when campaign metrics are stale, the Manager adds M08 Request evidence and re-runs M03, which the tactic record logs as deviations. On the right, a later version of L04 encodes that judgment as a typed branch with a bound of one retry, then rejoins the main path. The example is illustrative; real branches will come from real deviation patterns.

Continuity rules

1. **Stage Lines compile unchanged.** Each stage becomes its Moves as parallel Skill nodes followed by a Join; stages chain in order; the last node is *hand to Review* [P].
2. **Equivalence comes first.** The compiled form must behave identically to the stage runner before anything else changes (section 6).
3. **Branches come from evidence.** A branch is proposed when tactic records show the same deviation (same stage, same trigger, same added Move) recurring for a launch type [P; S: HA C10].

4. **A branch makes a new version.** L04 becomes L04 v2; v1 remains the baseline it is compared against (section 7).
5. **IDs stay stable.** Line and Move IDs do not change, so tactic records before and after compilation join cleanly.
6. **Managers can still deviate.** A branched Line reduces deviations; it does not forbid them, and every E3 step still waits for Sponsor approval [S: HA §5].

3. The method grammar for launch testing

The grammar is ZMC's, applied only to launch tactics and to HS's business terms (§4). A Move's expected effect is a change in a Position: a claim's verification state, a blocker's state or a hypothesis's state. The compiler can reference only Moves in the approved repertoire, whose handlers HA already runs [P; S: HA §5, HR08].

Elements

Element	Meaning [S: ZMC §3; HS §4]	Launch-testing example
Skill	Typed executable capability	Analytics query inside M03; bounded synthesis inside M06
Move	Versioned intervention with eligibility, inputs, effect class, risks and possible transitions	M05 Day Zero dependency check
Line	Conditional progression of Moves ending at Review	L04 Live checkpoint, with a branch to M08 when metrics are stale
Method	A family of alternative Lines for one launch goal	*Validate the message*: L02 when assumptions are unverified, L03 when a variant needs field signal
Protocol	Bounds, timeouts and failure rules	One evidence request per blocker; a test stops at its preregistered stop rule
Play / Playbook	Launch-type view of candidate and qualified Lines [S: HS §4]	*New-market launch*: L01 qualified, L02 candidate
Rubric	Typed evaluation question	*Is this a delivery defect or a strategy signal?* (G1 rubric) [S: HS §5.2]
Gate	Fails closed	Sponsor approval and mandate channel before any E3 Move

Element	Meaning [S: ZMC §3; HS §4]	Launch-testing example
Policy	Persistent constraint	"A Position contains no success forecast" [S: HA HR03]

Moves and the transitions they can declare

Moves	Effect	Can move
M01 Capture customer asset	E0	A claim to *reported*
M02 Bounded source research	E1, E2	A claim to *reported* or *contested*; never above *reported* on synthesis alone
M03 Verify metric identity and freshness	E1	An observation to *observed*, or *stale*
M04 Completeness check, M05 Day Zero dependency check	E0	A blocker to *open* or *cleared*
M06 Synthesize assumptions and gaps, M07 Propose variants	E2	Nothing directly; proposes findings and test candidates
M08 Request evidence	E0	An assumption to *awaiting evidence*
M09 Draft Test Plan	E0	A hypothesis to *planned*
M10 Dispatch approved test	E3	A hypothesis to *running*
M11 Record observation and Delta	E1	A hypothesis to *supported*, *not supported* or *inconclusive*, with attribution notes
M12 Escalate to the Sponsor	E0	None; hands the decision to the Sponsor

The Moves are HA's candidate set [S: HA §5]; the state names follow HS's evidence labels (§8) and are proposals for the domain owner to confirm [O]. Transitions are possibilities the compiler checks, not success rates [S: ZMC §3].

Effect classes and gates

Class	Gates	Available
E0 Internal	Policy gate	Yes

Class	Gates	Available
E1 Read external	Connector scope in the mandate (data-rights scope 1)	Yes
E2 Inference	Declared Skill; versioned prompt; provider data policy [O]	Yes, never with D4 personal data
E3 External effect	Data-rights scope 2; Sponsor approval of the Test Plan; authority re-checked at dispatch; idempotency key	Yes, behind an approved Test Plan only

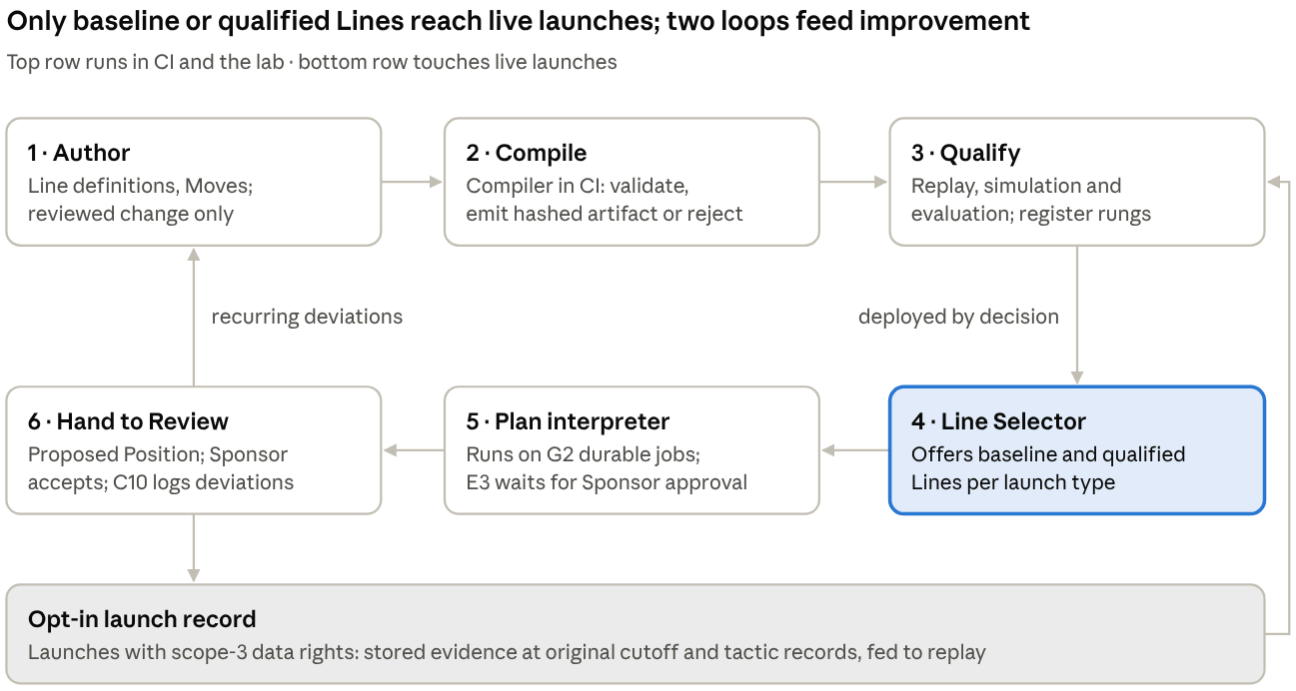
These are HA's classes and gates [S: HA §5, §9]. Hipstraw has no E4.

Negative repertoire

Every Line declares exclusions, and the Line Selector enforces them [S: ZMC §3]. Examples [P]: L03 is excluded where data-rights scope 2 grants no channel; L04 is excluded before Day Zero; L02 is excluded where the core assumptions already carry observed evidence; any Line with M10 is excluded while the Test Plan budget is unset.

4. Components and end-to-end flow

The components follow ZMC, placed in HA's processes. Two feedback loops drive the cycle: launches with scope-3 data rights feed replay and evaluation, and tactic records tell the domain owner which Lines need branches. Neither loop involves a model choosing anything [P].



Compiler end to end · 6 steps, 2 feedback loops

Only the bottom row touches live launches, and it uses only Lines the register marks as baseline or qualified. The left loop is how recorded Manager judgment becomes Line design; the right loop is how candidates earn their rungs.

Components and where they run

Component	Responsibility	Runs in	Replaces or extends
Line definitions	Versioned Lines, including G0 stage Lines	Repository, reviewed change	Extends HA Line definitions
Move registry and Skill Catalogue	Approved Moves, their Skills, effect classes and transitions	Generated at build	Extends HA Move registry
Compiler	Parse, resolve, build, validate, emit or reject	CI	Replaces HA Line lint
Compiled Artifact Store	Immutable artifacts and compilation reports	Object store plus SQL index	New
Qualification Register	Baseline flag, rung, deployment status, launch type	SQL; recorded decisions only	New

Component	Responsibility	Runs in	Replaces or extends
Line Selector	Offers baseline and qualified Lines per launch type	API, Position & Review module	Extends HA Line choice
Plan interpreter and plan step consumer	Advance compiled Lines event by event	Shared modules; job runner on G2 durable jobs	Replaces HA stage runner (section 6)
Replay Runner, simulator and evaluator	Compare candidates with baselines on scope-3 launches, including JEV/Vichara evaluations	Lab profile	New at G3 [S: HS §9]

Graph model, resolver, structural checks and interpreter hold no Hipstraw types; Move transitions, effect classes and scope rules are Hipstraw domain rules. That keeps G4 extraction possible [P; S: HS §9 G4].

5. Compilation pipeline and validation rules

The pipeline is ZMC's six deterministic stages, run in CI on every change to a Line definition [S: ZMC §5]. Hipstraw's rule set adds **authority** and **honesty** rules that make HA's boundaries enforceable at build time: a Line that could publish without approval, or claim more than its evidence supports, cannot compile [P].

Stages

1. **Parse** Line, Move, Method, Protocol and Playbook definitions against pinned schemas. G0 stage Lines parse unchanged (section 2).
2. **Resolve** every Move ID, Skill, gate, rubric and policy against the Skill Catalogue and Move registry.
3. **Build** the typed graph: Skill, Rubric, Gate, Branch, Join and Checkpoint nodes [S: ZMC §5].
4. **Validate** against the rules below.
5. **Emit** an immutable, hashed artifact with its compilation report.
6. **Reject** with reasons on any failure, keeping the input and report.

HA's Line lint retires when the compiler arrives; its rules are a subset of these [P].

Validation rules

Group	Rule	Rejects when	Basis
Structure	Known references	A Move, Skill, gate, rubric or policy is unknown or unversioned	[S: ZMC §5]
Structure	Typed edges	An output type does not match its bound input	[S: ZMC §5]
Structure	Bounded cycles and timeouts	A loop has no bound, or a wait has no timeout	[S: ZMC §5; HS §7]
Structure	Failure paths	A node can fail with no declared path	[S: ZMC §5]
Structure	Deterministic branches	A predicate is not a typed expression over claim, blocker, hypothesis or observation states	[S: HS §8.1; P]
Authority	Approval before effect	An E3 node is not preceded by a Test Plan approval gate and a mandate channel check	[S: HA §5, §9]
Authority	Idempotent effects	An E3 node has no idempotency key or reconciliation path	[S: HS §7]
Authority	Budget declared	A spending node has no budget and stop rule from its Test Plan	[S: HS §5.1]
Safety	Data classes	A node passes D4 personal data to an E2 node	[P, from HA §9]
Honesty	Achievable transitions	A Move declares a transition its Skills cannot produce	[P]
Honesty	Model outputs stay low	An E2 node raises a claim above *reported*	[P, from HS §8]
Honesty	Delta is not proof	A node marks a Delta causal, or sets a hypothesis to *supported* without a metric definition and stop rule	[S: HS §4, §12]
Scope	Approved Moves only	A node references a Move outside the repertoire approved under HR08	[P, from HA §5]

Group	Rule	Rejects when	Basis
Scope	Positions only	A node writes a success forecast or probability; the output schema has no such field	[S: HA HR03]
Scope	Review is the exit	Any path ends other than at *hand to Review*	[S: HA §5]
Governance	Exclusions present	A Line declares no negative-repertoire conditions	[S: ZMC §3]
Governance	Qualified dependencies	A Line uses a Move not qualified for its intended use	[S: ZMC §5]

What an artifact contains

The graph; resolved versions of every Move and Skill; effect and data classes; declared transitions; applicability and exclusions; bounds, timeouts and budgets; the source Line version (including its G0 stage form, if any); compiler version; and content hash [P]. The hash joins the mechanism stamp on every Review and Test Plan that uses the artifact [S: HA C8].

6. Running compiled Lines

Compiled Lines run inside HA's TestPlan, Action and TacticRecord machines: plan state sits beside the Test Plan and advances one committed event at a time through a plan interpreter in the shared modules [P; S: ZMC §6]. By G3, G2's durable jobs, triggers and recovery already carry work; the interpreter uses them as transport, while the Line's logic lives only in the compiled artifact [P; S: HA §10]. Something already runs Lines, the stage runner, so replacing it is a governed mechanism change, not an installation [S: HA §11].

Replacing the stage runner

1. **Compile every G0 Line unchanged.** Each stage Line compiles to a linear graph with parallel joins (section 2).
2. **Equivalence test in shadow.** For live launches, the interpreter runs the compiled Line beside the stage runner without executing effects. Both must produce the same jobs in the same stage order, with the same gate decisions.
3. **Decision record and canary.** On a clean result, the interpreter takes a limited share of new Test Plans, with rollback kept [S: HS §10, §12].
4. **Retire the stage runner.** Old Test Plans keep their mechanism stamp showing the stage runner; nothing is relabelled [S: HA C8].

5. **Only then add branches.** Branched versions are new Line versions that start on the qualification ladder (section 7).

This order separates two questions: does the new mechanism behave the same, and is a branched Line better? They are never tested together [P].

From Line choice to Line Selector

Stage	Who chooses	What is offered
HA G0	Market or Campaign Manager proposes; Sponsor accepts the Test Plan	All approved Lines not excluded for the launch
HA G1	Same, with a recorded default per launch type	Same, default first
G3	Manager or a recorded rule; a model may summarize alternatives but never chooses [S: HS §8.1; P]	Only Lines qualified and active for the launch type, plus consented trial assignments

If no compiled Line is eligible, the Test Plan falls back to Manager-directed Moves, recorded as such [P]. Sponsor approval of every E3 step is unchanged at every stage [S: HA §9].

Plan state, interim states and deviations

- **Plan state** (node states, loop counters, timeouts, plan version) commits with each event; there is no separate checkpoint store [S: ZMC §6].
- **Interim states** let branches read current claim, blocker and observation states mid-run. They are never shown as an accepted Position; the successor Position is proposed from all evidence at the end and committed only on acceptance [S: HA §8].
- **Deviations** (skip a node, add an approved Move, stop the plan) remain Manager commands, written to the same tactic record as at G0 [S: HA C10]. A falling deviation rate after branching is one sign a branched Line is working.

Failures

HA's failure handling applies: undeclared failures stop the plan and hand to Review; gate denials and revoked approvals stop the branch; loop bounds and timeouts take their declared exits; uncertain external effects are reconciled, never blindly repeated [S: HA §8; HS §11.3]. One addition: if an equivalence test fails in shadow, the stage runner stays in service and the difference is filed as a defect, not a decision [P].

7. Qualification and governance

The ladder follows ZMC [S: ZMC §7]. Lines already in production use when the compiler arrives become the **baseline**: they stay selectable, are never described as qualified, and every new or branched Line must beat them to climb [P]. Hipstraw adds one rung, simulated trials, because HS names them for G3; simulation results never count as field evidence [S: HS §9].

Two kinds of Line at G3

Kind	Origin	Deployment	Qualification
Baseline Line	A G0 stage Line compiled unchanged and shown equivalent (section 6)	Active, marked *baseline*	None claimed. Its tactic-record history is observational and biased by Manager choice [S: HA §11]
Candidate Line	A branched version, a new Line, or a candidate-repertoire proposal	Not selectable until deployed by decision	Climbs the ladder against the baseline for its launch type

Evidence and approver per rung

Rung	Evidence	Approver [P]
Generated	Authored or proposed	Author
Statically compiled	Section 5 rules pass	Compiler, automatic
Evaluated	Replay on scope-3 launches at their original evidence cutoff; JEV/Vichara evaluation where qualified [S: HS §9]	Evaluation stage, reviewed
Lab-qualified	Beats the baseline on held-out launches of the same launch type: missed blockers, evidence-supported findings, correction rate, cost [S: HS §10, §12]	Domain owner
Simulated	Simulated runs of the candidate and baseline on the same cases; informs the field trial, proves nothing about the field	Domain owner

Rung	Evidence	Approver [P]
Controlled field experimental	Shadow, then consented assignment between candidate and baseline [O: disclosure, HM13]	Sponsor and security owner
Field-qualified	Prospective decision quality, deviation rate, time-to-signal and cost for a stated launch type; campaign outcomes with stated confounders [S: HS §9.1, §12]	Domain owner and QA

Rules

- No rung is climbed automatically; every promotion has a decision record [S: HS §12; ZMC §7].
- Safety, authority and scope are pass/fail: a Line that acts without approval, touches excluded data or writes a success forecast is ineligible [S: HA §2, HR03].
- Qualification is per launch type and within stated applicability bounds; lab and simulated results are never called field-proven [S: HS §9, §9.1].
- A qualified candidate replaces the baseline for its launch type only by decision; the old baseline is restricted, not deleted, so history and rollback remain [P].

8. Alternatives and trade-offs

With staged Lines already running, the real choice is how far to extend what exists. The recommended path keeps the stage runner until a typed graph has proved it behaves identically, and adds branches only after that [P]. Each option faces HS's comparative test: same authorized cases, same evidence cutoff, measured on precision, missed blockers, correction rate, latency, cost and recoverability [S: HS §10].

Option	Strengths	Weaknesses for launch testing	Verdict
A. Keep stage Lines forever	Already built; simple; Managers handle branches	Recurring judgments stay manual; no controlled comparison; deviation rates may never fall	Right until entry condition 2 holds; a valid permanent outcome [S: HS §10]

Option	Strengths	Weaknesses for launch testing	Verdict
B. Add branches to the stage runner	Small change to running code	Grows an untyped mini-language without the validator; repeats the compiler badly	Reject
C. Compile stage Lines, prove equivalence, then add branches	One validator; authority and scope enforced at build; separates "same behaviour" from "better Line"	Two migrations instead of one; equivalence testing effort	**Recommended**
D. Line logic in a durable workflow engine	Multi-day waits, retries and recovery built in [S: HS §10]	Logic in engine code defeats static authority checks; a second source of truth beside the Test Plan	Use only as G2 transport under the interpreter, never as Line logic
E. A model plans launch steps	Flexible	Unbounded tool and action loops are a non-goal; no provenance; authority cannot be checked [S: HS §1.2, §8]	Reject

Trade-offs inside option C

Choice	Taken	Alternative	Why
Status of G0 Lines	Baseline, unqualified but selectable	Force them up the ladder	Avoids halting live launches; keeps claims honest
Authority and scope enforcement	Compile-time rules plus the dispatch-time gate	Runtime checks only	An unauthorized Line can never be deployed, and the gate still re-checks at the point of effect

Choice	Taken	Alternative	Why
Equivalence before branching	Required	Branch during migration	Separates mechanism risk from method risk
Comparison unit	Launch type	All launches	Matches how Lines are actually chosen; reduces selection bias
Simulation	A rung below field trials	Accept simulation as qualification	HS forbids claiming field effectiveness from simulation [S: HS §9]

9. Decision register

Fourteen decisions. They sit under HA's HR01–HR12 and must not contradict them; HA wins on conflict [P].

ID	Decision	Section	Approver	Status
HM01	Build only when the six entry conditions hold, measured from tactic records	1	Product + domain owner	Needs approval
HM02	Compile launch tactics only; never campaign execution, channel or budget management workflows	1	Product owner	Needs approval
HM03	G0 stage Lines compile unchanged to linear graphs	2	Architecture owner	Proposed
HM04	Replace the stage runner through shadow equivalence, a decision record and a canary	6	Architecture owner	Proposed
HM05	Add branches only after equivalence, as new Line versions	2, 6	Domain owner	Proposed
HM06	Enforce authority, honesty and scope rules at compile time, in addition to the dispatch-time gate	5	Architecture + security owner	Proposed

ID	Decision	Section	Approver	Status
HM07	A compiled Line's only output is a proposed successor Position for human acceptance; no success forecast; a Delta is never causal proof	5	Product owner	Needs approval
HM08	G0 Lines become the unqualified baseline per launch type	7	Domain owner	Needs approval
HM09	Qualify Lines per launch type against that type's baseline; simulation never counts as field evidence	7	Domain owner + QA	Proposed
HM10	The Line Selector grows from Line choice; a Manager or recorded rule chooses, and a model may only summarize alternatives	6	Product + domain owner	Proposed
HM11	Retire HA's Line lint when the compiler is in service	5	Architecture owner	Proposed
HM12	Approve the Move transition table and state names in section 3	3	Domain owner	Needs approval
HM13	Disclosure and consent for controlled field trials	7	Product + security owner	Open
HM14	The plan interpreter uses G2 durable jobs as transport; Line logic lives only in compiled artifacts	6, 8	Architecture owner	Proposed

10. Open questions and dependencies

Most of what this spec needs comes from operating HA's G0 to G2. The questions below can be answered only from tactic records and launches with scope-3 data rights, not from design work [P].

#	Question	Blocks	Owner
MQ1	What deviation rate, sustained over how many launches, counts as "a real choice" (condition 2)?	HM01	Domain owner
MQ2	How many scope-3 launches per launch type are enough to compare a candidate with its baseline?	Condition 4; lab rung	Domain owner
MQ3	Which campaign-outcome measures allow valid comparison beyond judgment quality, and with what sample-size guardrails?	Condition 3; HS ADR-07	Product + domain owner
MQ4	Are the Move transitions and state names in section 3 right, once the repertoire comes from the Hipstraw Market Mastery workbook (HA HQ1)?	HM12	Domain owner
MQ5	What exactly must match for stage-runner equivalence: jobs, order, gate decisions, timing?	HM04	Architecture owner + QA
MQ6	Which launch types does the Line Selector distinguish?	Selection; qualification unit	Domain owner
MQ7	Disclosure and consent for controlled field trials, including what the Sponsor is told	HM13	Product + security owner
MQ8	Protocol bounds: evidence-request limits, test durations, re-test limits and checkpoint waits	Protocols	Domain owner

Dependencies

- [ ] HA running with complete tactic records (C10) on every Test Plan.
- [ ] Launches with scope-3 data rights accumulating per launch type.
- [ ] HA G1: rubrics, Delta reconciliation and a recorded default Line per launch type.
- [ ] HA G2: durable jobs, triggers and recovery validated with live integrations.
- [ ] HA HR08 approved: the first-release repertoire, drawn from the workbook (HQ1).
- [ ] Answers to MQ1–MQ6 before G3 design starts.